

Creating comprehensive study notes from PDF documents ▾

File System and Disk Scheduling

PDF

You are an expert teacher and study-note creator.

Read the entire PDF carefully from the first page to the last page.

Create comprehensive study notes in very simple Bangla and English so that a beginner can understand everything without reading the original PDF.

Requirements:

1. Do NOT skip any important topic, concept, definition, formula, diagram explanation, example, or conclusion.

Show more

PDF Study Notes: File System and Disk Scheduling

(সহজ বাংলা ও ইংরেজিতে — Beginner Friendly)

PART 1: FILE SYSTEM (ফাইল সিস্টেম)

Topic 1: What is a File System? (ফাইল সিস্টেম কী?)

Easy Explanation:

ফাইল সিস্টেম হলো ডিস্কে ফাইল ও ফোল্ডার সাজানোর একটা পদ্ধতি। মনে করো তোমার বাসায় একটা বড় আলমারি আছে — সেই আলমারিতে কোন জিনিস কোথায় আছে সেটা যদি একটা তালিকায় লেখা থাকে, সেটাই ফাইল সিস্টেমের মতো কাজ করে।

In simple English: A file system is how an Operating System (OS) organizes files and folders on a disk — like a well-organized filing cabinet.

Important Points:

- OS-এ একটা বা একাধিক ফাইল সিস্টেম থাকতে পারে
- File system-এর দুটো মূল দিক আছে:
 1. **Data Structures** — ডিস্কে data ও metadata সাজানোর পদ্ধতি
 2. **System Call Implementation** — `open`, `read`, `write` এর মতো কমান্ড কীভাবে কাজ করে
- ডিস্ক সাধারণত **512 bytes** এর ব্লক হিসেবে কাজ করে
- ফাইল সিস্টেম এই ব্লকগুলোতে ফাইল সাজায়

 **Exam Tip:** "File system-এর দুটো main aspect কী?" — এটা প্রায়ই পরীক্ষায় আসে।

 Topic 2: Simple File System Structure (সিম্পল ফাইল সিস্টেমের গঠন)

Easy Explanation:

একটা সহজ ফাইল সিস্টেমে ডিস্ককে কয়েকটা অংশে ভাগ করা হয়:

```
[S][i][d][Inode blocks...][Data blocks (D)...]
0  7  8                15 16 ..... 63
```

Diagram Description: ডিস্কটিকে নম্বর দেওয়া ব্লকে ভাগ করা হয়েছে। প্রথমে Superblock (S), তারপর inode bitmap (i), data bitmap (d), তারপর inode blocks, তারপর data blocks।

চারটি মূল অংশ:

অংশ	কাজ
Data Blocks	ফাইলের আসল ডেটা রাখে
Inode	প্রতিটা ফাইলের metadata রাখে (কোথায় ডেটা আছে, permission ইত্যাদি)
Bitmap	কোন inode/data block খালি আছে তা ট্র্যাক করে
Superblock	সব block সম্পর্কে master plan রাখে

Important Points:

- **Data blocks** → ফাইলের actual content এক বা একাধিক block-এ থাকে
- **Inode blocks** → প্রতিটা block-এ এক বা একাধিক inode থাকে
- **Bitmap** → 0 মানে ব্যবহৃত, 1 মানে খালি (বা উল্টো, implementation-এর উপর নির্ভর করে)
- **Superblock** → OS বুট করার সময় প্রথমে এটা পড়ে

✦ Topic 3: Inode Table (আইনোড টেবিল)

Easy Explanation:

Inode মানে **Index Node**। প্রতিটা ফাইলের জন্য একটা করে inode থাকে। এটা ফাইল সম্পর্কে সব তথ্য ধরে রাখে — কিন্তু ফাইলের আসল ডেটা না।

মনে করো inode হলো একটা **ফাইলের পরিচয়পত্র** — নাম, অনুমতি, ডেটা কোথায় আছে সব লেখা।

Inode-এ কী থাকে?

1. **File Metadata** — permission, access time, file size ইত্যাদি
2. **Pointers** — ফাইলের data block-এর disk block number

Inode Table Diagram:

Inode গুলো array হিসেবে সাজানো থাকে। Inode number = ঐ array-তে index।

Iblock 0 → inode 0, 1, 2, 3...

Iblock 1 → inode 16, 17, 18...

Iblock 2 → inode 32, 33, 34...

🔗 **Viva Question:** "Inode number কী?" → উত্তর: Inode array-তে ফাইলের inode-এর index বা position।

✦ Topic 4: Inode Structure — How Does Inode Track Blocks?

Easy Explanation:

ফাইলের ডেটা ডিস্কে পরপর (contiguous) থাকে না। তাই inode-কে বিভিন্ন block-এর নম্বর ট্র্যাক করতে হয়।

৩টি পদ্ধতি আছে:

পদ্ধতি ১: Direct Pointers (সরাসরি পয়েন্টার)

- ছোট ফাইলের জন্য
- inode নিজেই প্রথম কয়েকটা block-এর নম্বর ধরে রাখে
- Example: inode তে লেখা আছে "ফাইলের ডেটা block 5, 9, 12 তে আছে"

পদ্ধতি ২: Indirect Block (পরোক্ষ পয়েন্টার)

- বড় ফাইলের জন্য
- inode একটা **indirect block**-এর নম্বর রাখে
- সেই indirect block-এ actual data block-এর নম্বর থাকে

```
inode → [indirect block number]
      ↓
      indirect block → [data block 1, data block 2, ...]
```

পদ্ধতি ৩: Double & Triple Indirect Blocks

- আরও বড় ফাইলের জন্য
- Double indirect: inode → indirect block → আরেকটা indirect block → data blocks
- Triple indirect: তিন স্তর গভীর

এই পুরো ব্যবস্থাকে বলে **Multi-level Index**।

 **Important Exam Question:** "Multi-level index কী এবং কেন ব্যবহার করা হয়?"

Topic 5: File Allocation Table (FAT)

Easy Explanation:

FAT হলো block ট্র্যাক করার আরেকটা পদ্ধতি। এতে প্রতিটা disk block-এর জন্য একটা entry থাকে যেটা বলে "পরের block কোনটা"।

মনে করো ফাইল হলো একটা chain (শিকল) — FAT বলে দেয় শিকলের পরের কড়াটা কোথায়।

FAT কীভাবে কাজ করে:

- FAT-এ প্রতিটা disk block-এর জন্য একটা entry আছে
- Entry-তে থাকে: পরের file block-এর নম্বর, অথবা NULL (যদি শেষ block হয়)
- প্রথম block-এর pointer থাকে inode-এ

```
inode → first block (e.g., block 5)
FAT[5] → 9 (next block)
FAT[9] → 12 (next block)
FAT[12] → NULL (end of file)
```

Inode vs FAT তুলনা:

বিষয়	Inode	FAT
Pointer কোথায়	inode-এ	আলাদা FAT table-এ
ব্যবহার	Linux/Unix	Windows (FAT32)

📌 Topic 6: Opening a File (ফাইল খোলা)

Easy Explanation:

`open()` system call কেন দরকার? কারণ ভবিষ্যতে read/write করার জন্য inode মেমোরিতে রেডি রাখতে হয়।

Open করলে কী হয় — Step by Step:

Step 1: pathname (যেমন `/home/user/file.txt`) শুরু থেকে traverse করা হয়

Step 2: Root (`/`) এর inode আগে থেকেই জানা থাকে (OS এটা bootstrap করে)

Step 3: Recursively কাজ করে:

- Parent directory-র inode fetch করো
- তার data block পড়ো
- Child-এর inode number বের করো
- Child-এর inode fetch করো
- Path শেষ না হওয়া পর্যন্ত repeat করো

Step 4: যদি নতুন ফাইল হয়:

- Bitmap থেকে নতুন inode ও data block allocate করো
- Directory entry আপডেট করো

Return: `open()` একটা **file descriptor (fd)** নম্বর return করে যেটা in-memory inode-কে point করে

🔗 **Exam Question:** "`open()` system call-এর কাজ কী?"

📌 Topic 7: Open File Table (ওপেন ফাইল টেবিল)

Easy Explanation:

দুটো টেবিল থাকে:

১. Global Open File Table:

- সিস্টেমে যতগুলো ফাইল খোলা আছে (socket, pipe সহ) সবার জন্য একটা করে entry
- প্রতিটা entry in-memory inode-কে point করে

২. Per-Process Open File Table:

- প্রতিটা process-এর নিজের আলাদা টেবিল আছে
- এটা একটা array — **file descriptor number = এই array-তে index**
- এই per-process entry, global table-এর entry-কে point করে

Default File Descriptors:

প্রতিটা process শুরু থেকেই তিনটা ফাইল খোলা পায়:

fd নম্বর	নাম	কাজ
0	stdin	Standard Input (keyboard)
1	stdout	Standard Output (screen)
2	stderr	Standard Error (error messages)

Open System Call কী করে:

- Global table-এ একটা entry তৈরি করে
- Per-process table-এ একটা entry তৈরি করে
- fd নম্বর return করে

✦ Topic 8: Reading and Writing a File

Easy Explanation:

ফাইল পড়া বা লেখার সময় যা হয়:

Step-by-Step Process:

Step 1: File descriptor দিয়ে in-memory inode access করো

Step 2: Current read/write offset অনুযায়ী কোন data block দরকার তা বের করো

Step 3: ডিস্ক থেকে সেই block fetch করো এবং operation (read/write) করো

Step 4 (শুধু write-এর জন্য): নতুন block দরকার হলে bitmap ব্যবহার করে allocate করে

Step 5: inode-এ access time ও অন্য metadata আপডেট করে

 **Viva Question:** "Write operation-এ bitmap কেন দরকার?" উত্তর: নতুন ফাইল data রাখার জন্য খালি block খুঁজে বের করতে।

Topic 9: Virtual File System (VFS)

Easy Explanation:

বিভিন্ন ফাইল সিস্টেম (ext4, NTFS, FAT) আলাদাভাবে কাজ করে। কিন্তু Linux চায় একটাই সিস্টেম call দিয়ে সব কাজ হোক। এজন্য VFS ব্যবহার করা হয়।

VFS হলো একটা **মধ্যস্থতাকারী layer** — application ও actual file system-এর মাঝে।

VFS কীভাবে কাজ করে:

- VFS ফাইল সিস্টেমকে objects হিসেবে দেখে: files, directories, inodes, superblock
- এই objects-এ operations define করে (যেমন: directory-তে filename খোঁজো)
- System call logic VFS objects-এর উপর লেখা থাকে
- নতুন ফাইল সিস্টেম বানাতে হলে শুধু VFS object-এর functions implement করে এবং kernel-কে pointer দাও

সুবিধা:

System call implementation পরিবর্তন করতে হয় না — শুধু ফাইল সিস্টেমের নিচের implementation বদলালেই চলে।

```

Application (open, read, write)
    ↓
  VFS Layer
    ↓
[ext4] [NTFS] [FAT32] ← আলাদা implementations
  
```

 **Exam Question:** "VFS কেন ব্যবহার করা হয়?"

PART 2: DISK SCHEDULING (ডিস্ক শিডিউলিং)

Topic 10: What is Disk Scheduling?

Easy Explanation:

ডিস্কে অনেক request একসাথে আসে। FIFO (আসার ক্রমে) সার্ভ না করে, request গুলো **reorder** করা হয় যাতে disk head কম নড়াচড়া করে।

কেন? Disk head-এর movement কমালে:

- Seek time কমে
- Rotational delay কমে
- পারফরম্যান্স বাড়ে

কে করে? মূলত **disk controller** করে, কারণ OS disk-এর internal geometry জানে না।

সাধারণ Example (সব algorithm-এ একই ব্যবহার করা হবে):

- **Request Queue:** 98, 183, 37, 122, 14, 124, 65, 67
- **Head শুরু:** 53
- **Disk range:** 0-199

 Topic 11: FCFS — First Come First Served

Easy Explanation:

যে request আগে এসেছে, সেটা আগে serve করো। একদম সহজ, কিন্তু অদক্ষ।

Step-by-Step:

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Calculation:

Movement	Distance
53→98	45
98→183	85
183→37	146
37→122	85
122→14	108

Movement	Distance
14→124	110
124→65	59
65→67	2
Total	640

Diagram বর্ণনা:

হেড বারবার ডানে-বামে যাচ্ছে — অনেক unnecessary movement।

💡 **সমস্যা:** মোট head movement = **640 cylinders** — অনেক বেশি।

📌 Topic 12: SSTF — Shortest Seek Time First

Easy Explanation:

Current head position থেকে যে request-টা সবচেয়ে কাছে, সেটা আগে serve করো।

এটা CPU scheduling-এর SJF (Shortest Job First)-এর মতো।

Step-by-Step (head শুরু 53):

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

মোট head movement = **236 cylinders**

সমস্যা:

- **Starvation** হতে পারে — দূরের request কখনো serve নাও হতে পারে যদি কাছে কাছে request আসতেই থাকে

🔗 **Exam Question:** "SSTF-এ starvation কেন হয়?"

📌 Topic 13: SCAN Algorithm (Elevator Algorithm)

Easy Explanation:

Disk arm এক দিক থেকে অন্য দিকে যায় — যেতে যেতে requests serve করে। শেষে পৌঁছে উল্টো দিকে ফেরে।

এটাকে **Elevator Algorithm** বলে — লিফটের মতো কাজ করে।

Step-by-Step (head শুরু 53, প্রথমে উপরে যাচ্ছে):

53 → 65 → 67 → 98 → 122 → 124 → 183 → [শেষে 199] → 37 → 14

মোট head movement = **208 cylinders**

Diagram বর্ণনা:

হেড একদিকে সব serve করে, তারপর ফেরে — অনেক কম zigzag।

✅ **সুবিধা:** FCFS ও SSTF-এর চেয়ে ভালো

✳️ Topic 14: C-SCAN — Circular SCAN

Easy Explanation:

SCAN-এর মতোই, তবে পার্থক্য হলো:

- হেড এক দিক থেকে অন্য দিকে যায়, serve করতে করতে
- শেষে পৌঁছে সরাসরি শুরুতে ফিরে আসে (কোনো request serve না করে)
- তারপর আবার একই দিকে যায়

এটা cylinders-কে **circular list** হিসেবে treat করে।

কেন ভালো?

More uniform wait time — সব request-এর জন্য অপেক্ষার সময় প্রায় সমান।

Diagram বর্ণনা:

হেড ডানে গিয়ে 183 পর্যন্ত serve করে, তারপর 199-এ যায়, তারপর সরাসরি 0-তে ফিরে বাম দিকের requests (14, 37) serve করে।

✳️ Topic 15: C-LOOK

Easy Explanation:

C-SCAN-এর একটা উন্নত version। পার্থক্য:

- হেড ডিস্কের শেষ প্রান্ত পর্যন্ত যায় না
- শুধু শেষ request পর্যন্ত যায়, তারপর সরাসরি প্রথম request-এ ফিরে আসে

কেন ভালো?

অপ্রয়োজনীয় disk end পর্যন্ত যাওয়া বন্ধ — সময় বাঁচে।

Step-by-Step:

53 → 65 → 67 → 98 → 122 → 124 → 183 → (jump back) → 14 → 37

Diagram বর্ণনা:

183-তে শেষ হয়ে 199-এ না গিয়ে সরাসরি 14-তে ফিরে আসে।

 QUICK REVISION NOTES

 File System — One-Line Summary:

Topic	Summary
File System	ডিস্কে ফাইল সাজানোর পদ্ধতি
Data Block	ফাইলের আসল content
Inode	ফাইলের metadata + data block pointer
Bitmap	কোন block খালি সেটা ট্র্যাক করে
Superblock	সব block-এর master plan
Direct Pointer	inode সরাসরি data block point করে (ছোট ফাইল)
Indirect Pointer	inode → indirect block → data block (বড় ফাইল)
FAT	পরের block pointer chain আকারে রাখে
Open File Table	Global (সব process) + Per-process টেবিল
VFS	সব ফাইল সিস্টেমের জন্য common interface

 Disk Scheduling — Comparison Table:

Algorithm	Total Movement	সমস্যা
FCFS	640	সবচেয়ে বেশি movement
SSTF	236	Starvation হতে পারে
SCAN	208	শেষ প্রান্ত পর্যন্ত যায়
C-SCAN	কম	Uniform wait time
C-LOOK	সবচেয়ে কম	শেষ request পর্যন্তই যায়



MOST IMPORTANT EXAM TOPICS

- ✓ File system-এর চারটি মূল অংশ (Data Block, Inode, Bitmap, Superblock)
- ✓ Inode কী এবং এতে কী থাকে
- ✓ Direct, Indirect, Double Indirect pointer — পার্থক্য ও ব্যবহার
- ✓ FAT vs Inode পার্থক্য
- ✓ `open()` system call-এ কী হয়
- ✓ Global vs Per-process Open File Table
- ✓ fd 0, 1, 2 কী
- ✓ VFS কেন ব্যবহার করা হয়
- ✓ FCFS, SSTF, SCAN, C-SCAN, C-LOOK — সব algorithm-এর কাজ ও head movement
- ✓ SSTF-এ starvation সমস্যা
- ✓ Elevator algorithm (SCAN) — কেন এই নাম



VIVA QUESTIONS AND ANSWERS

Q1: Inode কী? A: Index Node — প্রতিটা ফাইলের metadata (permission, time, data block numbers) রাখে।

Q2: Superblock কী কাজ করে? A: ডিস্কের সব block-এর master plan রাখে — কোনগুলো inode, কোনগুলো data block।

Q3: `open()` কেন দরকার? A: inode মেমোরিতে এনে রাখার জন্য, যাতে পরের read/write দ্রুত হয়।

Q4: File Descriptor কী? A: Per-process open file table-এ index নম্বর যেটা opened file-কে identify করে।

Q5: Disk scheduling কেন করা হয়? A: Seek time ও rotational delay কমাতে, disk performance বাড়াতে।

Q6: SSTF-এ starvation কেন হয়? A: দূরের requests কখনো serve নাও হতে পারে যদি কাছে কাছে request আসতেই থাকে।

Q7: SCAN ও C-SCAN-এর পার্থক্য কী? A: SCAN ফেরার পথেও serve করে, C-SCAN শুধু এক দিকেই serve করে — তারপর শুরুতে ফিরে আসে।

Q8: C-LOOK ও C-SCAN-এর পার্থক্য? A: C-SCAN disk-এর শেষ প্রান্ত পর্যন্ত যায়, C-LOOK শুধু শেষ request পর্যন্ত যায়।

Q9: VFS কী? A: Virtual File System — OS ও বিভিন্ন file system implementation-এর মাঝের abstraction layer।

Q10: Multi-level index কী? A: Inode-এ direct, indirect, double indirect এবং triple indirect pointer ব্যবহার করে বিভিন্ন আকারের ফাইল handle করার পদ্ধতি।

GLOSSARY OF TERMS

Term	সহজ সংজ্ঞা
File System	ডিস্কে ফাইল সাজানোর OS পদ্ধতি
Block	ডিস্কের সবচেয়ে ছোট storage unit (সাধারণত 512 bytes)
Inode	একটা ফাইলের সব তথ্য (metadata) যে structure-এ রাখা হয়
Bitmap	কোন block/inode ব্যবহৃত বা খালি তা দেখানোর bit map
Superblock	ফাইল সিস্টেমের overall layout তথ্য
Direct Pointer	inode সরাসরি data block দেখায়
Indirect Block	data block-এর নম্বর ধারণ করা আলাদা block
FAT	File Allocation Table — পরের block-এর চেইন
File Descriptor	Open file-এর integer identifier
VFS	Virtual File System — সব FS-এর common interface
Seek Time	Disk head correct cylinder-এ যেতে যে সময় লাগে
FCFS	First Come First Served scheduling
SSTF	Shortest Seek Time First

Term	সহজ সংজ্ঞা
SCAN	Elevator algorithm — এক দিক থেকে অন্য দিকে
C-SCAN	Circular SCAN — একমুখী, তারপর শুরুতে ফিরে
C-LOOK	C-SCAN-এর মতো কিন্তু শেষ request পর্যন্তই যায়
Starvation	কোনো request অনেকক্ষণ serve না হওয়া
Metadata	ফাইল সম্পর্কে তথ্য (content নয়, যেমন size, permission)

💡 **Final Tip:** এই notes পড়লে exam-এ File System ও Disk Scheduling-এর সব প্রশ্নের উত্তর দিতে পারবে। Algorithm গুলোর diagram মাথায় রাখো এবং total head movement মনে রাখো।

File System da kmn black box er moto mane bujte partici na akdom core vabe

ফাইল সিস্টেম — একদম ভেতর থেকে বোঝো 🧠

আগে একটা real life example দিয়ে শুরু করি

তোমার কাছে একটা বিশাল খাতা আছে — ৬৪ পাতার।

এই খাতায় তুমি গল্প লিখবে, রেসিপি লিখবে, অঙ্ক করবে।

কিন্তু সমস্যা হলো — খাতায় কোথায় কী লেখা আছে কীভাবে মনে রাখবে?

এই সমস্যা সমাধানের জন্য তুমি একটা সিস্টেম বানালে।

এই সিস্টেমটাই হলো File System।

খাতার সিস্টেমটা কেমন হবে?

তুমি খাতার পাতাগুলো ভাগ করলে এভাবে:

- পাতা ১ → Master Plan (কোন পাতায় কী আছে)
- পাতা ২ → খালি/ভরা চিহ্ন (inode এর জন্য)

পাতা ৩ → খালি/ভরা চিহ্ন (content এর জন্য)

পাতা ৪-১০ → পরিচয়পত্র রাখার জায়গা

পাতা ১১-৬৪ → আসল লেখার জায়গা

ডিস্কেও **হুবহু একই কাজ** হয়। শুধু "পাতা" বলার বদলে বলে "block"।

এখন ধাপে ধাপে প্রতিটা অংশ

● Block — সবকিছুর শুরু এখান থেকে

ডিস্ক মানে একটা বিশাল জায়গা।

OS এই জায়গাকে **ছোট ছোট সমান টুকরোয়** ভাগ করে।

প্রতিটা টুকরো = **১টা Block** (সাধারণত 4KB)

ডিস্ক = [Block 0][Block 1][Block 2][Block 3]...[Block 63]

এখন এই blocks গুলোকে **কাজ অনুযায়ী** ভাগ করা হয়।

● Superblock — খাতার প্রথম পাতা

মনে করো খাতার **একদম প্রথম পাতায়** লেখা আছে:

- Block 0 → এটা Master Plan (Superblock)
- Block 1 → এটা inode খালি/ভরার চিহ্ন
- Block 2 → এটা data খালি/ভরার চিহ্ন
- Block 3-7 → এগুলো পরিচয়পত্র (inodes)
- Block 8-63 → এগুলো আসল ডেটা

Superblock হলো এই প্রথম পাতা।

OS যখন চালু হয়, **সবার আগে** Superblock পড়ে — কারণ এটা না পড়লে বাকি কিছুই খুঁজে পাবে না।

● Bitmap — খালি সিট কোনটা?

সিনেমা হলের কথা ভাবো।

টিকেট কাউন্টারে একটা **চার্ট** আছে:

সিট ১ →  ভরা
 সিট ২ →  খালি
 সিট ৩ →  ভরা
 সিট ৪ →  খালি

Bitmap হলো এই চার্ট।

দুটো bitmap থাকে:

- **inode bitmap** → কোন inode slot খালি আছে
- **data bitmap** → কোন data block খালি আছে

নতুন ফাইল বানাতে গেলে OS bitmap দেখে খালি জায়গা খোঁজে।

🔵 Inode — ফাইলের পরিচয়পত্র

এটাই সবচেয়ে গুরুত্বপূর্ণ।

প্রতিটা ফাইলের জন্য একটা করে পরিচয়পত্র থাকে। এই পরিচয়পত্রই হলো **inode**।

পরিচয়পত্রে কী থাকে?

ফাইলের inode (পরিচয়পত্র)
Size: 5KB
Owner: তুমি
Permission: read/write
Created: আজকে
Last accessed: ৫ মিনিট আগে
ডেটা আছে:
Block 15 এ
Block 22 এ
Block 31 এ

একটা জিনিস খেয়াল করো:

inode-এ ফাইলের আসল লেখা নেই।

শুধু আছে — "আসল লেখা কোথায় আছে।"

এটা ঠিক যেমন তোমার পরিচয়পত্রে তুমি নিজে থাকো না — শুধু তোমার ঠিকানা থাকে।

● Data Block — আসল ফাইলের ডেটা

এখানেই ফাইলের আসল content থাকে।

Block 15 → "আমার গল্পের প্রথম অংশ..."

Block 22 → "...মঝের অংশ..."

Block 31 → "...শেষের অংশ।"

এখন সব মিলিয়ে একটা গল্প

তুমি `poem.txt` নামে একটা ফাইল বানালে।

ডিস্কে কী হলো:

১. OS bitmap চেক করলো → inode slot 5 খালি পেলো
২. OS bitmap চেক করলো → data block 18, 25 খালি পেলো
৩. inode 5 এ লিখলো:
"ডেটা আছে block 18 আর 25 এ"
৪. Block 18 এ লিখলো poem এর প্রথম অংশ
৫. Block 25 এ লিখলো poem এর দ্বিতীয় অংশ
৬. Bitmap আপডেট করলো → inode 5, block 18, 25 ভরা দেখালো

ফাইল delete করলে কী হয়:

inode 5 খালি করলো (bitmap এ )
block 18, 25 খালি করলো (bitmap এ )
আসল ডেটা কিন্তু মুছে না! শুধু "খালি" চিহ্ন দেওয়া হয়।

এই কারণেই deleted file recovery software কাজ করে! 😱

Inode এর ভেতরে Pointer — এটা কেন দরকার?

ধরো তোমার একটা ছোট ফাইল আছে — মাত্র ৩টা block জুড়ে।

inode সরাসরি বলে দিতে পারে:

```

inode → block 5, block 9, block 12
      ↓       ↓       ↓
    [data]   [data]   [data]

```

এটা **Direct Pointer** সহজ।

কিন্তু যদি ফাইল **অনেক বড়** হয়? হাজারটা block?

inode-এর ভেতরে এত জায়গা নেই সব block নম্বর রাখার।

তখন কী করে?

```

inode → একটা "সাহায্যকারী block" এর নম্বর রাখে
      ↓
    সেই block এ → [block 5, block 9, block 12, block 18...]
                  ↓           ↓
                [data]     [data]

```

এই "সাহায্যকারী block" = **Indirect Block**

আরও বড় ফাইলের জন্য দুই স্তর, তিন স্তর — **Double/Triple Indirect**

ফাইল খোলা (open) — ভেতরে কী হয়?

তুমি লিখলে: `open("/home/user/poem.txt")`

OS এর মাথায় শুধু "/" (root) এর inode নম্বর জানা আছে।

বাকিটা এভাবে খোঁজে:

```

Step 1: "/" এর inode পড়লো
Step 2: "/" এর data block পড়লো
        → দেখলো "home" folder আছে, তার inode নম্বর 3
Step 3: inode 3 পড়লো (home এর)
Step 4: home এর data block পড়লো
        → দেখলো "user" folder আছে, তার inode নম্বর 7
Step 5: inode 7 পড়লো (user এর)
Step 6: user এর data block পড়লো
        → দেখলো "poem.txt" আছে, তার inode নম্বর 14
Step 7: inode 14 মেমোরিতে নিয়ে এলো ✅
Step 8: তোমাকে দিলো → fd = 3

```

এখন `fd = 3` মানে "poem.txt এর inode মেমোরিতে রেডি আছে।"

পরের সব read/write এই fd দিয়েই হবে — আর pathname দিয়ে search করতে হবে না।

Open File Table — দুটো টেবিল কেন?

তোমার process Global Table
(per-process table)

fd 0 → stdin —————→ keyboard inode

fd 1 → stdout —————→ screen inode

fd 2 → stderr —————→ screen inode

fd 3 → poem.txt —————→ poem.txt inode ←— অন্য process ও
একই ফাইল খুলতে পারে

কেন দুটো টেবিল?

কারণ দুটো আলাদা process একই ফাইল খুলতে পারে।

Global table এ **একটাই** entry — inode একবারই memory তে আসে।

দুটো process এর fd আলাদা, কিন্তু দুটোই global table এর **একই** entry point করে।

VFS — শেষ টপিক, সহজে বলি

Windows এ NTFS, Linux এ ext4, USB তে FAT32।

তিনটা আলাদা file system। কিন্তু তুমি সব জায়গায় একই `open()`, `read()`, `write()` লেখো।

কীভাবে সম্ভব?

VFS হলো একটা **translator**।

তোমার code: `open("file.txt")`

↓

VFS

↙ ↓ ↘

ext4 NTFS FAT32

VFS বলে — "ফাইল open করার কাজটা যেভাবেই করো, আমাকে এই নিয়মে করো।"

প্রতিটা file system সেই নিয়ম মেনে নিজের মতো implement করে।

✱ এক ছবিতে সব মিলিয়ে

তুমি লিখলে: `read(fd=3, buffer, 100)`

↓

fd 3 → per-process table

↓

global open file table

↓

poem.txt এর inode (memory তে)

↓

inode দেখলো: offset এ data আছে block 22 তে

↓

ডিস্ক থেকে block 22 আনলো

↓

তোমার buffer এ দিলো ✅

এখন কোন অংশটা এখনো ঘোলা লাগছে বলো — সেটা আরো ভেঙে দেখাবো! 😊

Open Theke last

Open File Table থেকে শেষ পর্যন্ত — একদম ভেতর থেকে

আগে মনে করো কোথায় ছিলাম

তুমি `open("/home/user/poem.txt")` করলে।

OS খুঁজে খুঁজে poem.txt এর inode পেলো।

এখন কী হবে?

Open File Table — দুটো টেবিল

প্রথমে একটা সমস্যার কথা ভাবো

তুমি একটা ফাইল খুললে।

তোমার বন্ধুও একই ফাইল খুললো।

দুজনের কাছেই কি আলাদা আলাদা inode copy থাকবে memory তে?

না। সেটা memory waste।

inode একবারই memory তে আসবে। দুজনেই সেটা share করবে।

এই সমস্যা সমাধানের জন্যই দুটো টেবিল।

টেবিল ১ — Global Open File Table

সিস্টেমে এখন যতগুলো ফাইল খোলা আছে — সবার একটা করে entry এখানে।

Global Open File Table

Entry	কী আছে
0	keyboard এর inode (stdin)
1	screen এর inode (stdout)
2	screen এর inode (stderr)
3	poem.txt এর inode ← তুমি
4	poem.txt এর inode ← বন্ধু

খেয়াল করো — poem.txt এর **inode একটাই** memory তে।

দুটো entry আছে কিন্তু দুটোই **একই inode** কে point করছে।

টেবিল ২ — Per-Process Open File Table

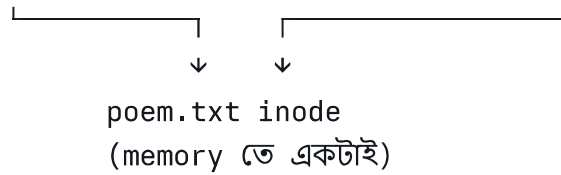
প্রতিটা process এর নিজের আলাদা টেবিল।

তোমার process এর টেবিল

fd	কোথায় যায়
0	Global[0]
1	Global[1]
2	Global[2]
3	Global[3]

বন্ধুর process এর টেবিল

fd	কোথায় যায়
0	Global[0]
1	Global[1]
2	Global[2]
3	Global[4]



fd মানে হলো — per-process টেবিলে **index** নম্বর।

তুমি `read(fd=3, ...)` লিখলে মানে — "আমার টেবিলের ৩ নম্বরে যা আছে সেটা পড়ো।"

Default তিনটা fd — সবসময় খোলা থাকে

প্রতিটা process জন্মের সাথেই তিনটা ফাইল খোলা পায়।

fd 0 → stdin → তুমি keyboard এ যা টাইপ করো
 fd 1 → stdout → screen এ যা দেখাও (print)
 fd 2 → stderr → error message যায় এখানে

এই তিনটা আগে থেকেই আছে। তুমি `open()` না করলেও।

তাই তোমার প্রথম `open()` করলে fd পাও 3 থেকে।

Reading a File — ভেতরে কী হয়

তুমি লিখলে:

```
c
read(fd=3, buffer, 100);
// fd=3 থেকে 100 bytes পড়ো, buffer এ রাখো
```

Step by step কী হয়:

Step 1:

fd=3 দিয়ে per-process টেবিলে গেলো
 → পেলো Global Table এর entry 3

Step 2:

Global Table entry 3 দেখলো
 → পেলো poem.txt এর inode (memory তে)

Step 3:

inode দেখলো current read offset কত
(ধরো এখন offset = 0, মানে শুরু থেকে পড়বো)

Step 4:

inode এর pointer দেখলো
→ offset 0 মানে Block 18 এ আছে

Step 5:

ডিস্ক থেকে Block 18 আনলো

Step 6:

Block 18 এর ডেটা buffer এ দিলো 

Step 7:

inode এ "last accessed time" আপডেট করলো
offset এগিয়ে দিলো (0 → 100)

পরের বার `read()` করলে offset 100 থেকে শুরু হবে।

Writing a File — ভেতরে কী হয়

c

```
write(fd=3, "নতুন লেখা", 50);
```

Step by step:

Step 1-3: একই - fd → Global Table → inode

Step 4:

inode দেখলো current write offset কত
→ ধরো offset = 200 (মানে এখানে লিখবো)

Step 5:

offset 200 কোন block এ পড়ে?
→ inode এর pointer দেখলো
→ Block 22 এ

Step 6:

যদি Block 22 আগে থেকে আছে → সেখানে লিখলো

যদি নতুন block দরকার হয়:

- data bitmap দেখলো কোন block খালি
- ধরো Block 31 খালি
- Block 31 allocate করলো
- inode এ Block 31 এর pointer যোগ করলো
- data bitmap এ Block 31 "ভরা" চিহ্ন দিলো

Step 7:

inode আপডেট করলো:

- file size বাড়লো
- last modified time বদলানো

Virtual File System (VFS) — একদম সহজে

সমস্যাটা আগে বোঝো

Linux এ অনেক রকম file system চলে একসাথে:

তোমার হার্ডডিস্ক → ext4
 USB drive → FAT32
 Windows partition → NTFS

তুমি যেখানেই ফাইল খোলো, একই command লেখো:

```
c
open("file.txt")
read(fd, buffer, 100)
write(fd, data, 50)
```

কিন্তু ext4 আর FAT32 সম্পূর্ণ আলাদাভাবে কাজ করে।

তাহলে একই command কীভাবে সব জায়গায় কাজ করে?

VFS হলো মাঝখানের দোভাষী

তোমার code

↓

open() ← তুমি শুধু এটুকু জানো

↓

VFS ← এটা বোঝে তুমি কোন ডিস্কে আছে

↙ ↓ ↘

ext4 FAT32 NTFS ← এরা নিজেদের মতো কাজ করে

VFS বলে দেয় — "ফাইল খুলতে হলে এই নিয়মে function বানাও।"

ext4 বলে — "ঠিক আছে, আমি আমার মতো বানাচ্ছি।"

FAT32 বলে — "ঠিক আছে, আমি আমার মতো বানাচ্ছি।"

কিন্তু **interface একই।**

VFS Objects — চারটা জিনিস দেখে

১. Superblock object
→ পুরো file system সম্পর্কে তথ্য
২. Inode object
→ একটা নির্দিষ্ট ফাইল সম্পর্কে তথ্য
৩. Dentry object (Directory Entry)
→ ফোল্ডারের ভেতরে কী আছে
৪. File object
→ একটা খোলা ফাইল সম্পর্কে তথ্য

নতুন file system বানাতে চাইলে — এই চারটার জন্য functions লিখো, kernel কে pointer দাও।
ব্যস।

সব মিলিয়ে একটা পূর্ণ গল্প

তুমি terminal এ লিখলে:

```
bash
```

```
cat poem.txt
```

পর্দার আড়ালে কী হলো:

১. cat program চললো

২. `open("/home/user/poem.txt")` call হলো
 - OS "/" থেকে শুরু করে খুঁজলো
 - `poem.txt` এর inode 14 পেলো
 - inode 14 memory তে আনলো
 - Global table এ entry হলো
 - Per-process table এ `fd=3` হলো
 - `open()` return করলো 3
৩. `read(fd=3, buffer, 4096)` call হলো
 - `fd=3` → Global table → inode 14
 - inode দেখলো: data আছে Block 18, 22 তে
 - Block 18 ডিস্ক থেকে আনলো
 - buffer এ দিলো
 - offset এগালো
৪. `write(fd=1, buffer, bytes_read)` call হলো
 - `fd=1` মানে `stdout` (screen)
 - screen এ দেখালো 
৫. আর পড়ার কিছু নেই
 - `read()` return করলো 0
৬. `close(fd=3)` call হলো
 - per-process table থেকে entry সরলো
 - global table থেকে entry সরলো
 - inode memory থেকে সরলো (যদি আর কেউ না দেখছে)

✳ মাথায় গেঁথে নেওয়ার জন্য

ফাইল খোলা = inode memory তে আনো, fd দাও
 ফাইল পড়া = inode → block number → ডিস্ক থেকে আনো
 ফাইল লেখা = inode → block number → ডিস্কে লেখো
 (নতুন block লাগলে bitmap দেখো)
 ফাইল বন্ধ = টেবিল থেকে entry সরানো

fd = per-process টেবিলের index
 inode = ফাইলের পরিচয়পত্র + data কোথায় আছে
 bitmap = কোন block/inode খালি আছে
 VFS = সব file system এর মাঝের দোভাষী

এখন কোনটা এখনো আটকে আছে? 😊

Open theke er problem

কোন "Open theke" বলছে একটু বলো —

এই তিনটার কোনটা?

Open er suru dia bujao

open() — একদম শুরু থেকে

আগে একটা প্রশ্ন — open() কেন দরকার?

তুমি কি সরাসরি `read()` করতে পারো না?

পারো না।

কারণ `read()` করতে হলে OS কে জানতে হবে —

- ফাইলটা ডিস্কের কোথায়?
- কোন block এ ডেটা আছে?
- Permission আছে কিনা?

এই সব তথ্য আছে **inode** এ।

তাই আগে inode টা memory তে আনতে হবে।

এই কাজটাই করে `open()` ।

এখন ধরো একটা situation

তোমার ডিস্কে আছে:

```

/
├── home
│   ├── user
│       └── poem.txt

```

তুমি লিখলে:

```

c
open("/home/user/poem.txt")

```

OS এর কাছে এই মুহূর্তে শুধু একটাই জিনিস জানা আছে —

root (/) এর inode নম্বর।

বাকি সব খুঁজে বের করতে হবে।

Step by Step — OS কীভাবে খোঁজে

Step 1 — "/" এর inode পড়া

OS জানে root এর inode সবসময় **inode নম্বর 2।**

ডিস্ক থেকে inode 2 আনলো (root এর inode)

inode 2 এ লেখা আছে:

root (/) এর inode	
type: directory	
data আছে: Block 5 এ	

Step 2 — "/" এর data block পড়া

inode বললো data আছে Block 5 এ।

Block 5 আনলো। Block 5 হলো "/" এর directory।

Directory মানে হলো একটা **তালিকা**:

Block 5 এর ভেতরে:

নাম	inode নম্বর
home	7
etc	12
usr	15

OS দেখলো path এ পরের নাম হলো **"home"** → inode নম্বর 7

Step 3 — "home" এর inode পড়ো

ডিস্ক থেকে inode 7 আনলো

inode 7 এ লেখা আছে:

home এর inode
type: directory
data আছে: Block 9 এ

Step 4 — "home" এর data block পড়ো

Block 9 আনলো:

Block 9 এর ভেতরে:

নাম	inode নম্বর
user	11
guest	13

পরের নাম **"user"** → inode নম্বর 11

Step 5 — "user" এর inode পড়ো

ডিস্ক থেকে inode 11 আনলো

inode 11 এ লেখা আছে:

user এর inode
type: directory
data আছে: Block 14 এ

Step 6 — "user" এর data block পড়ো

Block 14 আনলো:

Block 14 এর ভেতরে:

নাম	inode নম্বর
poem.txt	23
notes.txt	25

পেয়ে গেলো! **poem.txt** → inode নম্বর 23

Step 7 — poem.txt এর inode পড়ো

ডিস্ক থেকে inode 23 আনলো

inode 23 এ লেখা আছে:

poem.txt এর inode
type: file
size: 200 bytes
permission: read/write
data আছে: Block 18, Block 22

Step 8 — inode memory তে রাখো, fd দাও

inode 23 কে **memory** তে রেখে দিলো।

Global open file table এ entry করলো।

Per-process table এ entry করলো।

তোমার process এর টেবিল:

fd 0 → stdin

fd 1 → stdout

fd 2 → stderr

fd 3 → poem.txt এর inode 23  ← নতুন entry

`open()` return করলো → **fd = 3**

পুরো journey এক ছবিতে

```
open("/home/user/poem.txt")
  ↓
inode 2 (/)
  ↓
Block 5 → "home এর inode = 7"
  ↓
inode 7 (home)
  ↓
Block 9 → "user এর inode = 11"
  ↓
inode 11 (user)
  ↓
Block 14 → "poem.txt এর inode = 23"
  ↓
inode 23 (poem.txt) → memory তে রাখো
  ↓
fd = 3 দাও 
```

একটা জিনিস মাথায় রাখো

এই পুরো খোঁজার কাজে OS বারবার ডিস্ক থেকে পড়লো।

প্রতিটা step এ একটা করে ডিস্ক access।

Path যত লম্বা → তত বেশি ডিস্ক access → তত slow।

এই কারণেই `open()` একবার করে inode memory তে এনে রাখে।

পরের `read()` `write()` তে আর এই লম্বা খোঁজাখুঁজি করতে হয় না।

এখন কি clear হলো? নাকি কোনো step এ আবার আটকে গেছো? 😊

Ar aktu easy kore bujao 😊

`open()` — একদম চায়ের দোকানের ভাষায় 😊

একটা real life situation

তুমি একটা বড় লাইব্রেরিতে গেলে।

বললে — "আমাকে 'poem' বইটা দাও।"

লাইব্রেরিয়ান কি সাথে সাথে দিতে পারবে?

পারবে না।

আগে খুঁজতে হবে।

লাইব্রেরিয়ান কীভাবে খোঁজে?

১. প্রথমে মেইন ক্যাটালগ দেখলো
→ "Fiction বিভাগে যাও"
২. Fiction বিভাগে গেলো
→ "বাংলা shelf এ যাও"
৩. বাংলা shelf এ গেলো
→ "poem বই এখানে আছে, নিয়ে আসছি"
৪. বই এনে দিলো ✅

ব্যস এটুকুই।

এখন computer এ একই জিনিস

তুমি লিখলে:

```
open("/home/user/poem.txt")
```

এই `/home/user/poem.txt` হলো বইয়ের ঠিকানা।

OS হলো লাইব্রেরিয়ান।

OS কীভাবে খোঁজে — একদম সহজে

তুমি বললে: `/home/user/poem.txt`

↓

OS ভাবলো: ঠিকানায় তিনটা ধাপ আছে

home → user → poem.txt

ধাপ ১

OS গেলো "/" এ (মূল দরজা)

দেখলো একটা তালিকা:

home → ৭ নম্বর ঘরে

etc → ১২ নম্বর ঘরে

usr → ১৫ নম্বর ঘরে

আমার দরকার "home" → ৭ নম্বরে যাও

ধাপ ২

OS গেলো ৭ নম্বর ঘরে (home)

দেখলো আরেকটা তালিকা:

user → ১১ নম্বর ঘরে

guest → ১৩ নম্বর ঘরে

আমার দরকার "user" → ১১ নম্বরে যাও

ধাপ ৩

OS গেলো ১১ নম্বর ঘরে (user)

দেখলো আরেকটা তালিকা:

poem.txt → ২৩ নম্বর ঘরে

notes.txt → ২৫ নম্বর ঘরে

পেয়ে গেছি! poem.txt → ২৩ নম্বরে ✓

ধাপ ৪

২৩ নম্বর ঘর থেকে poem.txt এর

সব তথ্য (inode) memory তে আনলো

তোমাকে দিলো একটা নম্বর → fd = 3

fd = 3 মানে কী?

এটা হলো টোকেন নম্বর 🗑️

যেমন ডাক্তারের চেম্বারে গেলে টোকেন পাও।

পরে শুধু টোকেন দেখালেই ডাক্তার বোঝে তুমি কে।

তুমি পরে লিখবে: read(3, ...)

OS বুঝবে: ও, ৩ নম্বর টোকেন মানে poem.txt

আর লম্বা ঠিকানা `/home/user/poem.txt` লিখতে হবে না।

কেন এই ঝামেলা?

একটাই কারণ 👉

open() একবার করো → লম্বা খোঁজাখুঁজি একবার

read() বারবার করো → আর খোঁজতে হয় না, fd দিলেই হয়

খোঁজাখুঁজি বারবার করলে slow হয়ে যেতো।

এক লাইনে মনে রাখো

```
open() = ফাইল খোঁজো + memory তে আনো + টোকেন দাও  
read() = টোকেন দেখাও + ডেটা নাও  
close() = টোকেন ফেরত দাও
```

এখন clear? 😊 নাকি আরো ভাঙবো?

